
SmartCliff
CAREER MOBILITY SOLUTIONS
We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

SmartCliff
Career Mobility Solutions

Test Execution Reports in Automated Testing

NOV, 2025

Contents

Introduction to Test Reporting

- Tracking and analyzing Selenium test results can be difficult without clear reports, slowing down issue identification and debugging.
- A structured reporting solution provides detailed insights with essential test data, improving test management and speeding up troubleshooting.
- While Selenium itself can run tests efficiently, but it does not provide any advanced reporting feature.

Introduction to Test Reporting

- Test reporting is the process of collecting, analyzing, and presenting test execution results in a readable, meaningful format for:
    - Testers
    - Developers
    - Test Managers
    - Business stakeholders

Why Test Reports Are Important in IT Projects?

In real projects:

- Managers do not read logs
- Stakeholders do not open IDE
- CI/CD pipelines require visual evidence of quality

Reports answer:

- How many tests ran?
- How many passed/failed?
- Which features failed?
- Why did they fail?
- Evidence (screenshots)

Introduction to Extent Reports

- Extent Reports is an open-source reporting library useful for test automation. It can be easily integrated with major testing frameworks like JUnit, NUnit, TestNG, Cucumber etc.
- These reports are HTML documents that depict results as pie charts. They also allow the generation of custom logs, snapshots, and other customized details.
- Once an automated test script runs successfully, testers need to generate a test execution report.
- While TestNG does provide a default report, they do not provide the details.
- Extent Reports Documentation: **https://extentreports.com/docs/versions/5/java/index.html**

Extent Reports

- Extent Reports in Selenium are a popular reporting tool used to generate detailed and interactive test execution reports with customizable features, such as logs, screenshots, and test results.

Key Characteristics of Extent Reports in Selenium:

- **Customizable Reports:** Allows users to customize the look and feel of the reports, including test status and details.
- **Interactive UI:** Provides an interactive and user-friendly interface to view test execution results.
- **Test Status Tracking:** Displays detailed information on passed, failed, and skipped tests.
- **Logs and Screenshots:** Supports adding logs and screenshots for better test debugging and analysis.
- **Supports Multiple Formats:** Reports can be generated in HTML, PDF, or other formats for easy sharing and archiving.

Setting Up ExtentSparkReporter

- Follow these simple steps to set Extent Reports in Selenium:

Step 1: Firstly, add the Extent Reports dependency to your project. If you are using Maven, you can add the following dependency to your pom.xml file:

```
<dependency>
        <groupId>com.aventstack</groupId>
        <artifactId>extentreports</artifactId>
        <version>5.1.2</version>
</dependency>
```

Setting Up ExtentSparkReporter

Step 2: After adding the dependency, start ExtentReports in your test script. You need to create an ExtentReports instance, mention a path for the report, and start an ExtentTest object for each test.

Step 3: You can log various information (such as pass, fail, or skip) using the methods provided by Extent Reports.

Step 4: After all tests have run, finalize the report using flush() to save all logs and generate the final HTML report, which you can download for later.

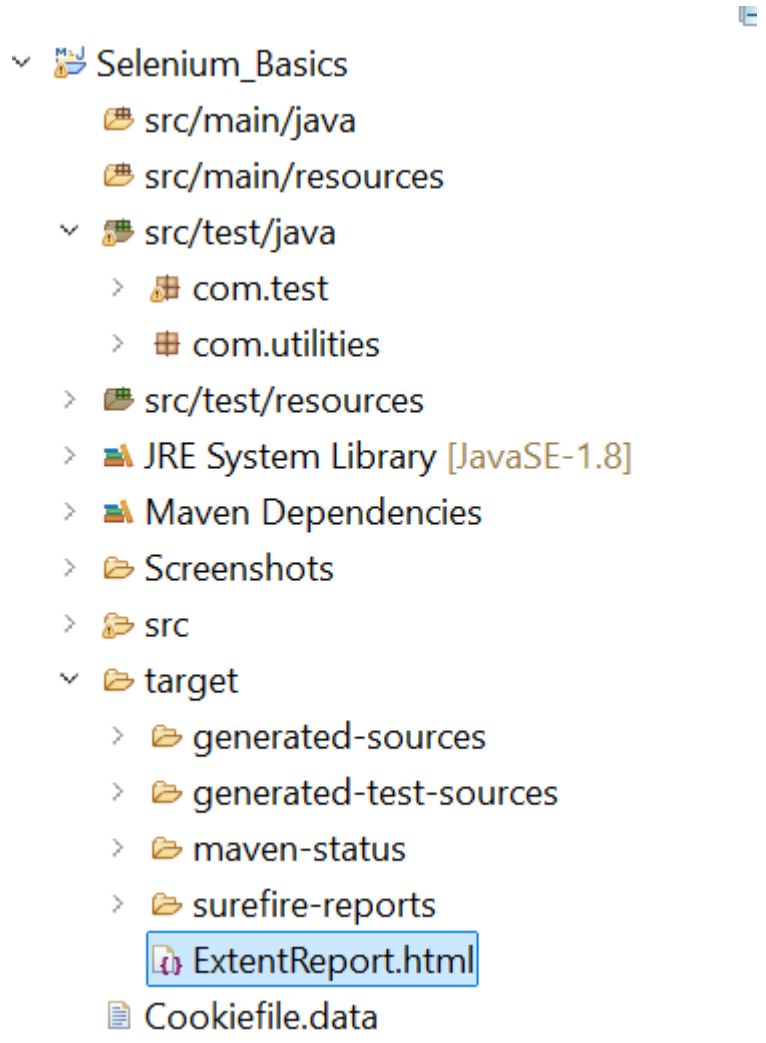
Example

```
import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.ExtentTest;
import com.aventstack.extentreports.reporter.ExtentHtmlReporter;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
public class TestReporting {
    public static void main(String[] args) {

        ExtentHtmlReporter htmlReporter = new ExtentHtmlReporter("extentReport.html");
        ExtentReports extent = new ExtentReports();
        extent.attachReporter(htmlReporter);
        WebDriver driver = new ChromeDriver();
        ExtentTest test = extent.createTest("Google Search Test");
```

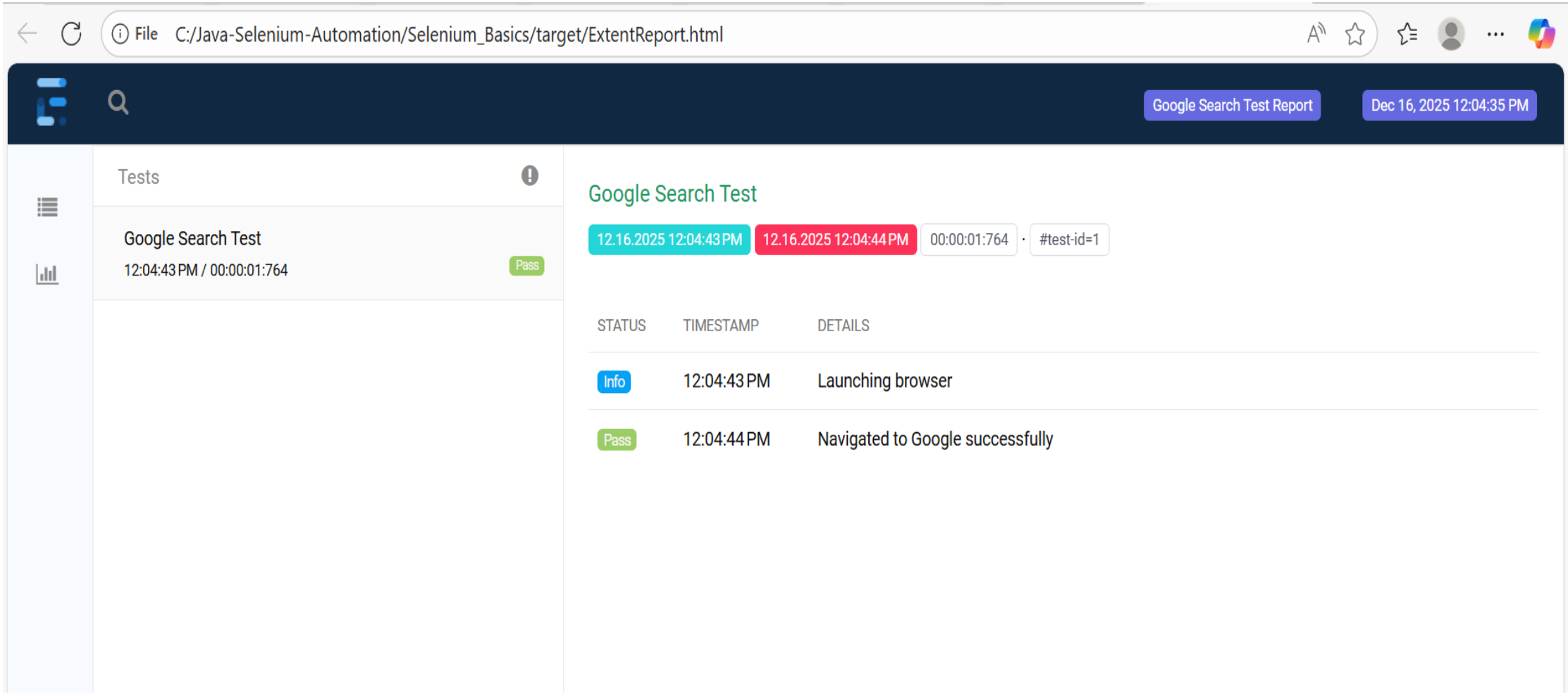
Example

```
try {
            driver.get("https://www.google.com");
            test.pass("Navigated to Google");
        } catch (Exception e) {
            test.fail("Test failed due to exception: " + e.getMessage());
        } finally {
            driver.quit();
            extent.flush(); // Save the report
        }
    }
}
```

Example – Check Report in Target Folder



Example – Right Click ExtentReport.html and open with in web browser



Report Attachments

- To add attachments, like screen images, two settings need to be added to the extent.properties.
- Firstly property, named screenshot.dir, is the directory where the attachments are stored.
- Secondly is screenshot.rel.path, which is the relative path from the report file to the screenshot directory.

extent.reporter.spark.out=Reports/Spark.html

screenshot.dir=/Screenshots/

screenshot.rel.path=../Screenshots/

Extent PDF Reporter

- The PDF reporter summarizes the test run results in a dashboard and other sections with feature, scenario, and, step details.
- The PDF report needs to be enabled in the extent.properties file.

extent.reporter.pdf.start=true

extent.reporter.pdf.out=PdfReport/ExtentPdf.pdf

Ported HTML Reporter

- The HTML reporter summarizes the test run results in a dashboard and other sections with feature, scenario, and, step details.
- The HTML report needs to be enabled in the extent.properties file.

extent.reporter.html.start=true

extent.reporter.html.out=HtmlReport/ExtentHtml.html

Customized Report Folder Name

- To enable the report folder name with date and\or time details, two settings need to be added to the extent.properties.
- These are basefolder.name and basefolder.datetimepattern. These will be merged to create the base folder name, inside which the reports will be generated.

basefolder.name=ExtentReports/SparkReport_

basefolder.datetimepattern=d_MMM_YY HH_mm_ss

Attach Image as Base64 String

- This feature can be used to attach images to the Spark report by setting the **src attribute of the img tag to a Base64** encoded string of the image.
- When this feature is used, no physical file is created. There is no need to modify any step definition code to use this.
- To enable this, use the below settings in extent.properties, which is false by default.

extent.reporter.spark.base64imagesrc=true

Environment or System Info Properties

- It is now possible to add environment or system info properties in the extent.properties or pass them in the maven command line.

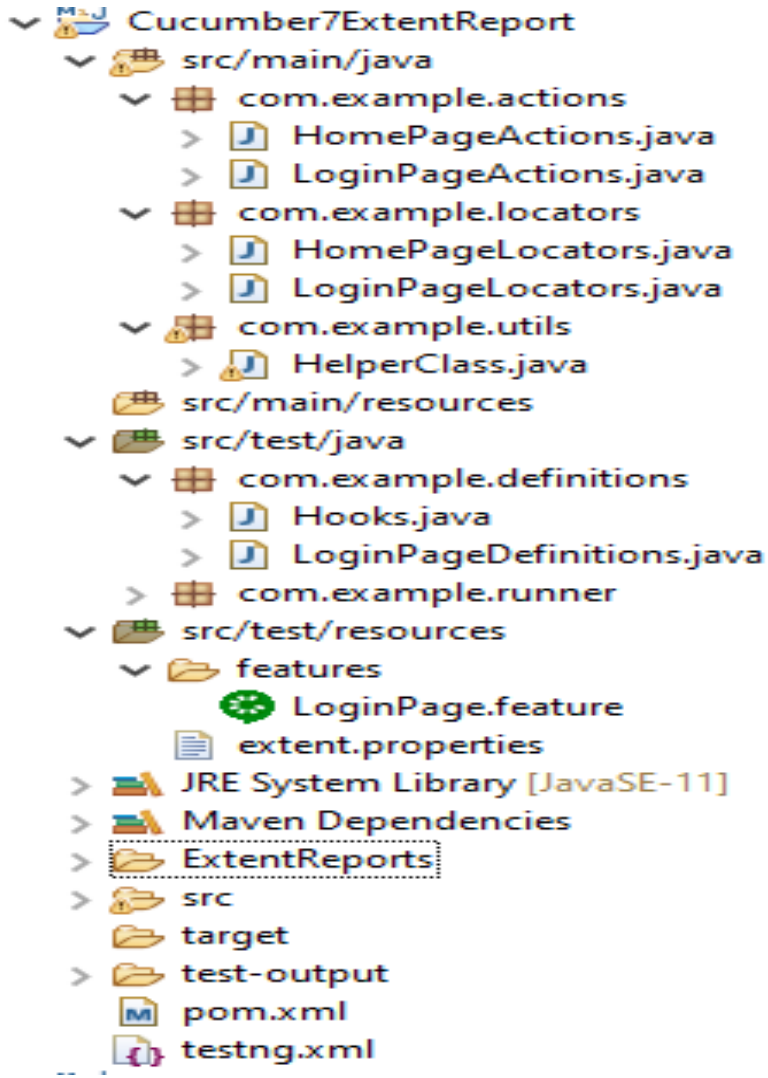
systeminfo.os=windows

systeminfo.version=10

Prerequisite

1. Java 8 or higher is needed for ExtentReport5
2. Maven or Gradle
3. JAVA IDE (like Eclipse, IntelliJ, or soon)
4. TestNG installed
5. Cucumber Eclipse plugin (in case using Eclipse)

Project Structure



Step 1: Add Maven dependencies to the POM

Add ExtentReport dependency

```
<dependency>
    <groupId>com.aventstack</groupId>
    <artifactId>extentreports</artifactId>
    <version>5.0.9</version>
</dependency>
```

Add tech grasshopper maven dependency for Cucumber

```
<dependency>
    <groupId>tech.grasshopper</groupId>
    <artifactId>extentreports-cucumber7-adapter</artifactI
    <version>1.7.0</version>
</dependency>
```

Step 2: Create a feature file in src/test/resources

- Create Simple feature file for checking a login in to any application.

```
Feature: Login to HRM Application
@ValidCredentials
Scenario: Login with valid credentials
Given User is on HRMLogin page "https://opensource-demo.orangehrmlive.com/"
When User enters username and password
Then User should be able to login sucessfully and new page open
```

Step 3: Create extent.properties file in src/test/resources

```
extent.reporter.spark.start=true
extent.reporter.spark.out=Reports/Spark.html

#PDF Report
extent.reporter.pdf.start=true
extent.reporter.pdf.out=PdfReport/ExtentPdf.pdf

#HTML Report
extent.reporter.html.start=true
extent.reporter.html.out=HtmlReport/ExtentHtml.html

#FolderName
basefolder.name=ExtentReports/SparkReport_
basefolder.datetimepattern=d_MMM_YY HH_mm_ss

#Screenshot
screenshot.dir=/Screenshots/
screenshot.rel.path=../Screenshots/

#Base64
extent.reporter.spark.base64imagesrc=true

#System Info
systeminfo.os=windows
systeminfo.version=10
```

Sample data.properties file in src/test/resources

```
database.baseUrl = https://localhost
database.port = 8080
database.username = admin
database.password = Admin123
```

data to the property file are in key-value pairs

Step 4: Create a Helper class in src/main/java

- Use Page Object Model with Cucumber and TestNG.
- Create a Helper class where we are
- Initializing the web driver,
- Initializing the web driver wait, defining the timeouts,
- and creating a private constructor of the class, it will declare the web driver,
- so whenever we create an object of this class, a new web browser is invoked.

Helper Class

```
public class HelperClass {
private static HelperClass helperClass;
private static WebDriver driver;
private static WebDriverWait wait;
public final static int TIMEOUT = 10;
HelperClass() {
// WebDriverManager.chromedriver().setup();
driver = new ChromeDriver();
wait = new WebDriverWait(driver, Duration.ofSeconds(TIMEOUT));
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(TIMEOUT));
driver.manage().window().maximize();
}
public static void openPage(String url) {
driver.get(url);
}
```

Helper Class

```
public static WebDriver getDriver() {
return driver;
}
public static void setUpDriver() {
if (helperClass==null) {
helperClass = new HelperClass();
}
}
public static void tearDown() {
if(driver!=null) {
driver.close();
driver.quit();
}
helperClass = null;
}
}
```

Step 5: Create Locator classes in src/main/java

- Create a locator class for each page that contains the detail of the locators of all the web elements.
- Create required locator classes
  - LoginPageLocators and HomePageLocators.

LoginPageLocators

```
package extentreport_demo;

import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class LoginPageLocators {
        @FindBy(name = "username")
          public WebElement userName;

          @FindBy(name = "password")
          public WebElement password;

          @FindBy(xpath =
"//*[@id='app']/div[1]/div/div[1]/div/div[2]/div[2]/form/div[3]/button")
          public WebElement login;

}
```

HomePageLocators

```
package extentreport_demo;

import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;

public class HomePageLocators {
        @FindBy(xpath = "//h6[text()='Dashboard']")
          public  WebElement homePageUserName;

}
```

Step 6: Create Action classes in src/main/java

- Create the action classes for each web page.
- These action classes contain all the methods needed by the step definitions.
- In this case, created 2 action classes – LoginPageActions and HomePageActions.

LoginPageActions

```
package extentreport_demo;

import java.io.File;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.util.Enumeration;
import java.util.Properties;

import org.openqa.selenium.support.PageFactory;

public class LoginPageActions {
        LoginPageLocators loginPageLocators = null;
        String strUserName, strPassword;

   public LoginPageActions() {

      this.loginPageLocators = new LoginPageLocators();

      PageFactory.initElements(HelperClass.getDriver(),loginPageLocators);
   }
```

LoginPageActions

```
 // Set user name in textbox
public void setUserName(String strUserName) {
   loginPageLocators.userName.sendKeys(strUserName);
}

// Set password in password textbox
public void setPassword(String strPassword) {
   loginPageLocators.password.sendKeys(strPassword);
}
  // Click on login button
public void clickLogin() {
   loginPageLocators.login.click();
}
   public void login() {
        File file = new File("C:\\\\Users\\\\Arun Kumar\\\\eclipse-
workspace\\\\extentreport_demo\\src\\test\\resources\\data1.properties");

                FileInputStream fileInput = null;
                try {
                        fileInput = new FileInputStream(file);
                } catch (FileNotFoundException e) {
                        e.printStackTrace();
                }
```

LoginPageActions

```
                Properties prop = new Properties();

                //load properties file
                try {
                        prop.load(fileInput);
                } catch (IOException e) {
                        e.printStackTrace();
                }

                strUserName = prop.getProperty("username");
                strPassword = prop.getProperty("password");

    // Fill user name
    this.setUserName(strUserName);

    // Fill password
    this.setPassword(strPassword);

    // Click Login button
    this.clickLogin();
  }
}
```

HomePageActions

```
import org.openqa.selenium.support.PageFactory;
public class HomePageActions {
HomePageLocators homePageLocators = null;
public HomePageActions() {
this.homePageLocators = new HomePageLocators();
PageFactory.initElements(HelperClass.getDriver(),homePageLocators);
}
// Get the User name from Home Page
public String getHomePageText() {
return homePageLocators.homePageUserName.getText();
}
}
```

Step 7: Create a Step Definition file in src/test/java

- Create the corresponding Step Definition file of the feature file.

LoginPageDefinitions

```
package extentreport_demo;

import org.testng.Assert;

import io.cucumber.java.en.Given;
import io.cucumber.java.en.Then;
import io.cucumber.java.en.When;

public class LoginPageDefinitions {
            LoginPageActions objLogin = new LoginPageActions();
   HomePageActions objHomePage = new HomePageActions();

   @Given("User is on HRMLogin page {string}")
   public void loginTest(String url) {

      HelperClass.openPage(url);

   }
```

LoginPageDefinitions

```
@When("User enters username and password")
public void goToHomePage() {

    // login to application
    objLogin.login();

    // go the next page

}


@Then("User should be able to login sucessfully and new page open")
public void verifyLogin() {

    // Verify home page
    Assert.assertTrue(objHomePage.getHomePageText().contains("Dashboard"));

}

}
```

Step 8: Create Hook class in src/test/java

- Create the hook class that contains the Before and After hook.
- @Before hook contains the method to call the setup driver which will initialize the chrome driver. This will be run before any test.
- @After to take screenshot and quit the browser.

SmartCliff
CAREER MOBILITY SOLUTIONS

Hooks Class

```
package extentreport_demo;

import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import extentreport_demo.HelperClass;
import io.cucumber.java.After;
import io.cucumber.java.Before;
import io.cucumber.java.Scenario;

public class Hooks{
    @Before
    public static void setUp() {
            HelperClass.setUpDriver();
    }

    @After
    public static void tearDown(Scenario scenario) {
        //validate if scenario has failed
        if(scenario.isFailed()) {
            final byte[] screenshot = ((TakesScreenshot)
            HelperClass.getDriver()).getScreenshotAs(OutputType.BYTES);
            scenario.attach(screenshot, "image/png", scenario.getName());
        }
        HelperClass.tearDown();
    }
}
```

Step 9: Create a Cucumber Test Runner class in src/test/java

- Add the extent report cucumber adapter to the runner class's CucumberOption annotation.

```
import io.cucumber.testng.AbstractTestNGCucumberTests;

import io.cucumber.testng.CucumberOptions;

@CucumberOptions(tags = "", features = "src/test/resources/features/LoginPage.feature", glue =
"com.example.definitions",

        plugin = {"com.aventstack.extentreports.cucumber.adapter.ExtentCucumberAdapter:"})


public class CucumberRunnerTests extends AbstractTestNGCucumberTests {

}
```

Step 10: Create the testng.xml for the project

- **Right-click on the project and select TestNG -> convert to TestNG.**

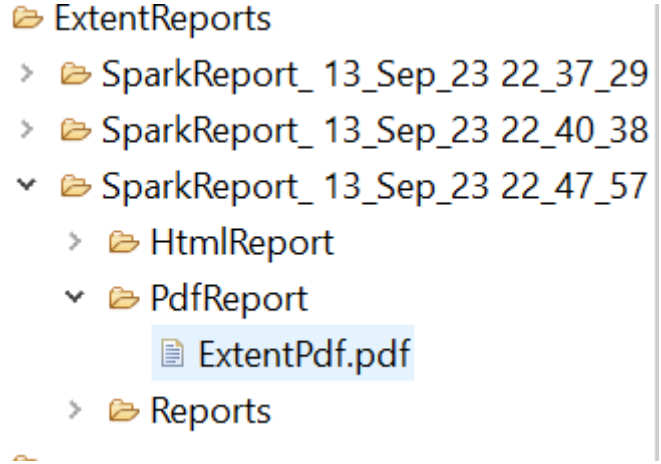
```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd"
<suite name="Suite">
  <test name="ExtentReport5 for Cucumber7">

  <classes>
  <class name = "com.example.runner.CucumberRunnerTests"/>
  </classes>
  </test> <!-- Test -->
</suite> <!-- Suite -->
```

Step 11: Execute the code

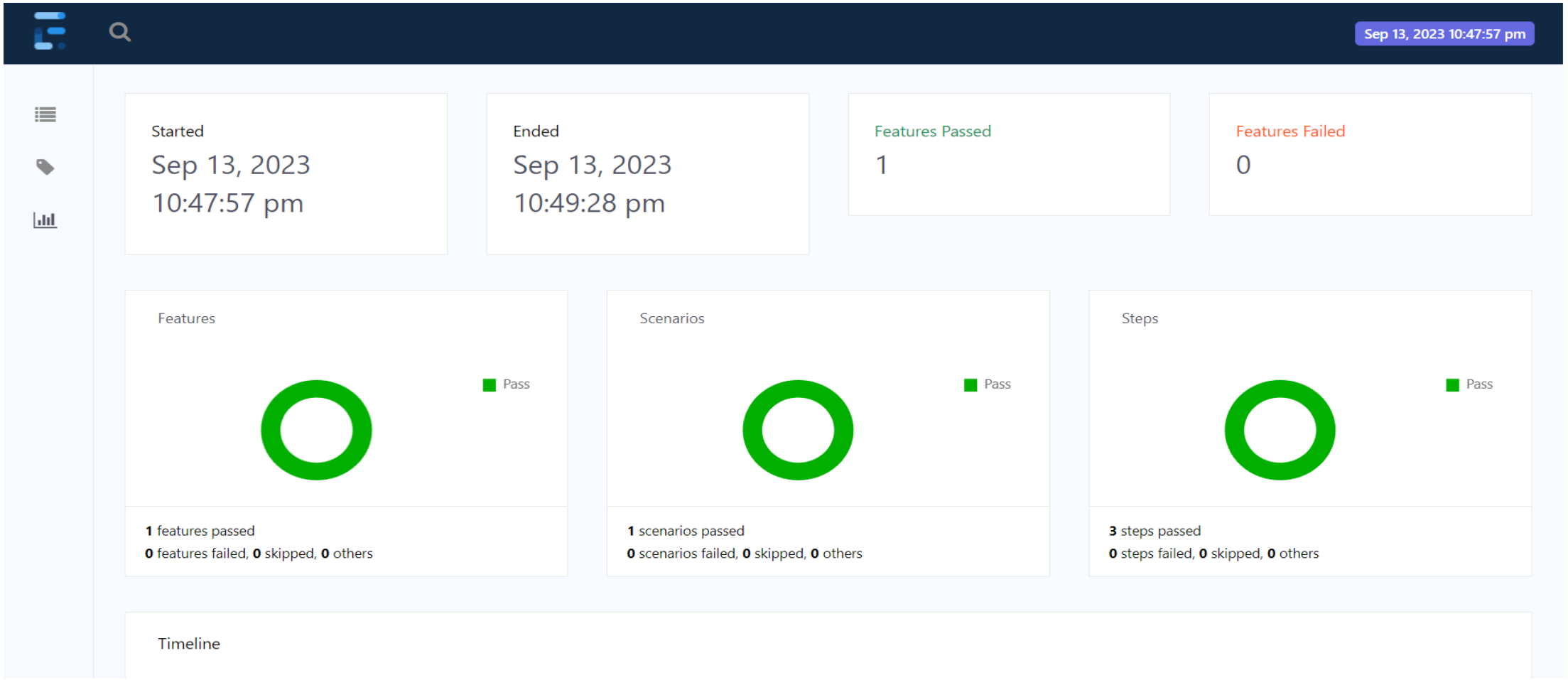
- Right Click on the Runner class and select Run As -> TestNG Test.

Step 12: View ExtentReport

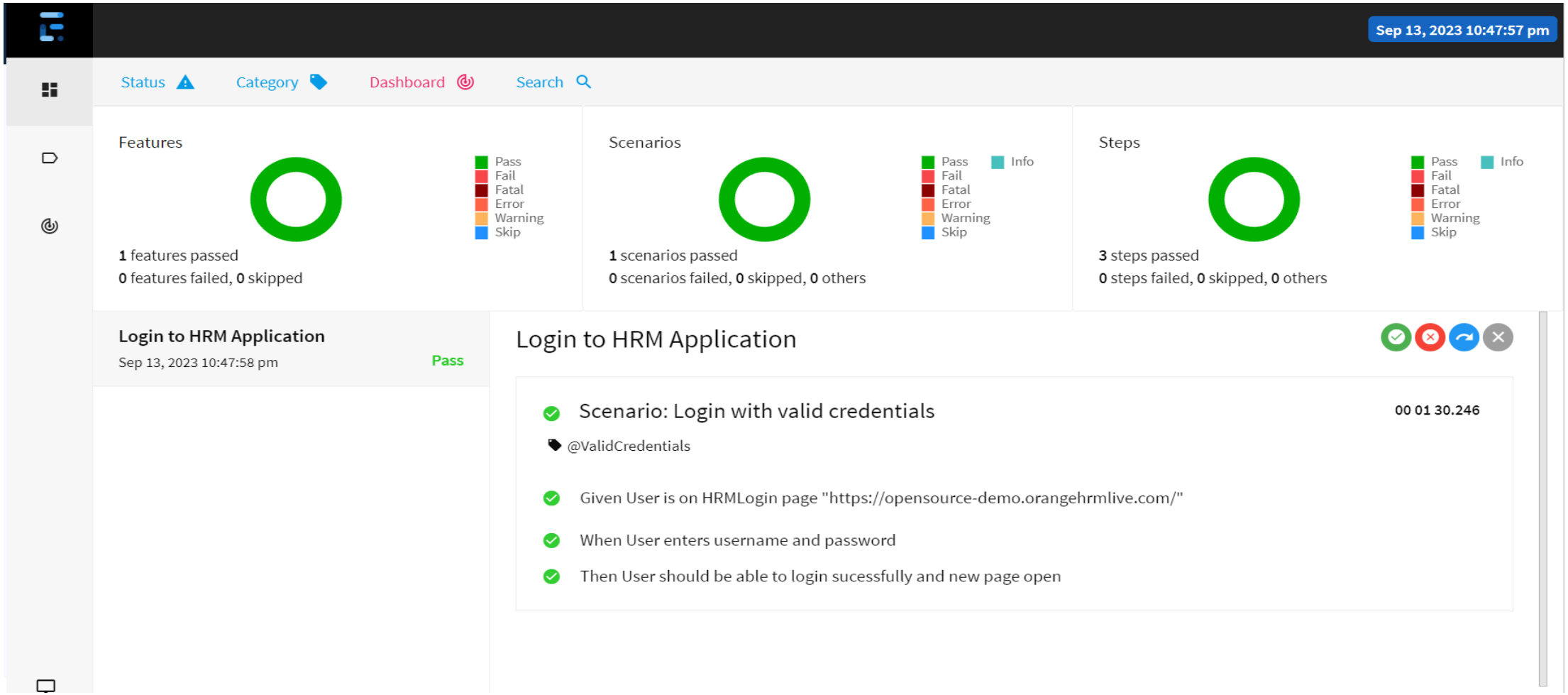
- Refresh the project and will see a new folder – SparkReport_ which further contains 4 folders – HtmlReport, PdfReport, Reports, and Screenshots.



Output


Sep 13, 2023 10:47:57 pm
Started
Sep 13, 2023
10:47:57 pm
Ended
Sep 13, 2023
10:49:28 pm
Features Passed
1
Features Failed
0
Features
Pass
1 features passed
0 features failed, 0 skipped, 0 others
Scenarios
Pass
1 scenarios passed
0 scenarios failed, 0 skipped, 0 others
Steps
Pass
3 steps passed
0 steps failed, 0 skipped, 0 others
Timeline

Output


Sep 13, 2023 10:47:57 pm
Status
Category
Dashboard
Search
Features
Pass
Fail
Fatal
Error
Warning
Skip
1 features passed
0 features failed, 0 skipped
Scenarios
Pass
Info
Fail
Fatal
Error
Warning
Skip
1 scenarios passed
0 scenarios failed, 0 skipped, 0 others
Steps
Pass
Info
Fail
Fatal
Error
Warning
Skip
3 steps passed
0 steps failed, 0 skipped, 0 others
Login to HRM Application
Sep 13, 2023 10:47:58 pm
Pass
Login to HRM Application
Scenario: Login with valid credentials
00 01 30.246
@ValidCredentials
Given User is on HRMLogin page "https://opensource-demo.orangehrmlive.com/"
When User enters username and password
Then User should be able to login sucessfully and new page open

Output


Menu
Acrobat Document
Create
Sign in
All tools
Edit
Convert
Sign
Find text or tools
Bookmarks
DASHBOARD
SUMMARY
TAGS
FEATURES
SCENARIOS
DETAILS
Cucumber Report
13-Sep-2023, 10:49:29 pm
Start : Sep 13, 10:47:58.257 pm
End : Sep 13, 10:49:28.510 pm
Duration : 1 m 30.253 s
Features
Scenarios
Steps
PASSED - 1
FAILED - 0
SKIPPED - 0
PASSED - 1
FAILED - 0
SKIPPED - 0
PASSED - 3
FAILED - 0
SKIPPED - 0
1
1
3
Pass %
100 %
1
6

Introduction to Allure Reporting Framework

- Allure Framework is a flexible lightweight multi-language test report tool that not only shows a very concise representation of what has been tested in a neat web report form, but allows everyone participating in the development process to extract the maximum useful information from the everyday execution of tests.
- Allure Report is a powerful reporting framework that provides interactive, easy-to-read test reports with detailed insights, including execution history, failure analysis, and attachments like screenshots and logs.

Introduction to Allure Reporting Framework

- With Allure, teams can enhance test visibility, improve debugging, and integrate seamlessly with popular automation test frameworks like JUnit, TestNG, Cucumber and Pytest.
- This guide explores Allure's key features and how to integrate it into your automation framework to generate advanced test reports effortlessly, capturing essential test details such as:
    - Test execution status (Passed, Failed, Skipped)
    - Step-by-step execution flow
    - Screenshots, logs, and attachments for debugging
    - Execution history and trends

Why Use Allure for Test Reporting?

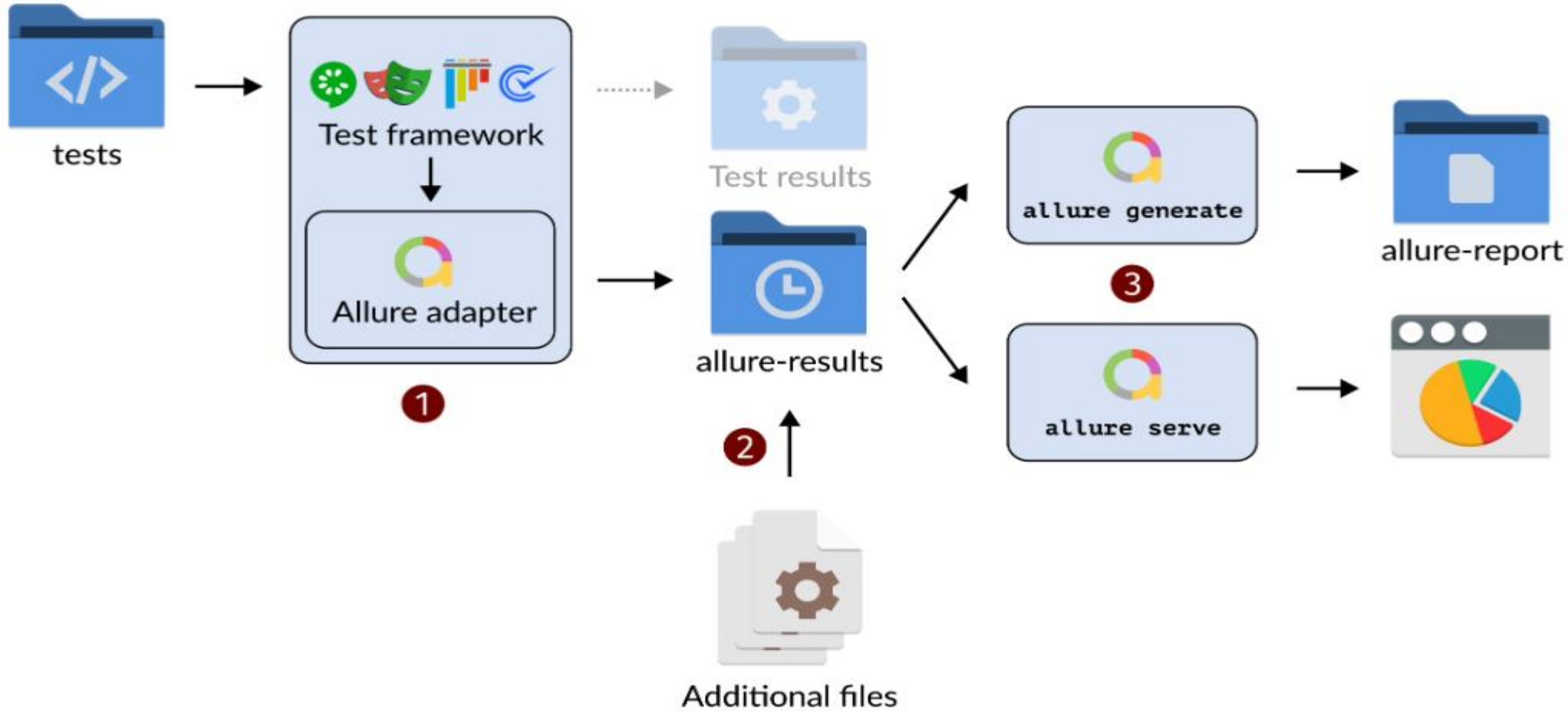
Allure Report enhances test reporting by providing clear, interactive, and data-rich insights into test executions. Here's why it stands out:

- **Visually Rich Reports:** Generates easy-to-understand, interactive reports with graphs, timelines, and stepwise execution details.
- **Detailed Test Insights:** Captures test steps, execution history, logs, and screenshots to aid debugging.
- **Seamless Integration:** Works with popular test frameworks like JUnit, TestNG, Pytest, and Cucumber without complex setup.
- **Execution History & Trend Analysis:** Helps track test trends over multiple runs to monitor stability and failures.

How Allure Report is generated?

- Allure is based on standard xUnit results output but adds some supplementary data. Any report is generated in two steps.
- During test execution (first step), a small library called adapter attached to the testing framework saves information about executed tests to XML files.
- We already provide adapters for popular Java, PHP, Ruby, Python, Scala, and C# test frameworks.
- During report generation (second step), the XML files are transformed into an HTML report. This can be done with a command line tool, a plugin for CI, or a build tool.

How Allure Report is generated?



Install Allure

Prerequisites for Allure Reporting

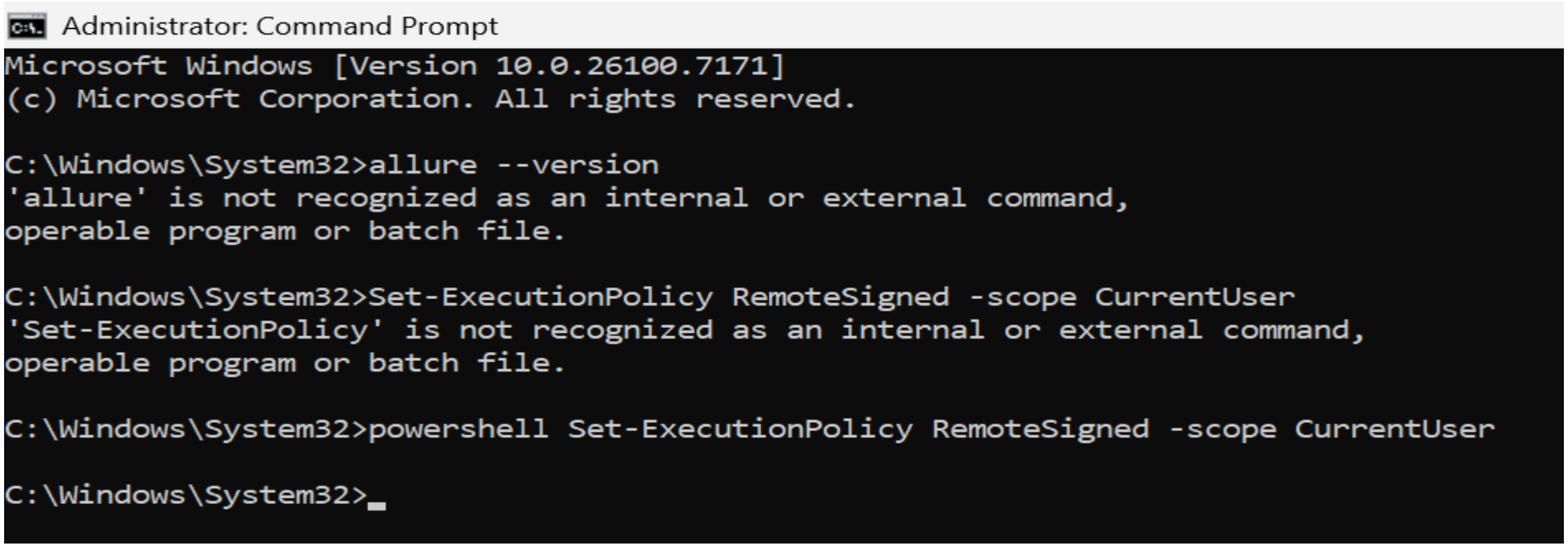
To set up Allure Reporting, you need to meet the following prerequisites:

- **Java Installation:** Ensure that Java version 8 or above is installed on your system. The Java directory should be specified in the JAVA_HOME environment variable.
- **Allure Installation:** Install the Allure command-line tool. This can be done using Scoop or Nodejs on Windows, Homebrew on macOS and Linux, or by downloading and unpacking the Allure archive.
- **Environment Variable Setup:** Add the path to the Allure bin directory to your system's PATH environment variable to ensure the allure command is accessible.

Install Allure

- For Windows, Allure is available from the Scoop command line installer.
- Run the following command in command prompt with administrator rights

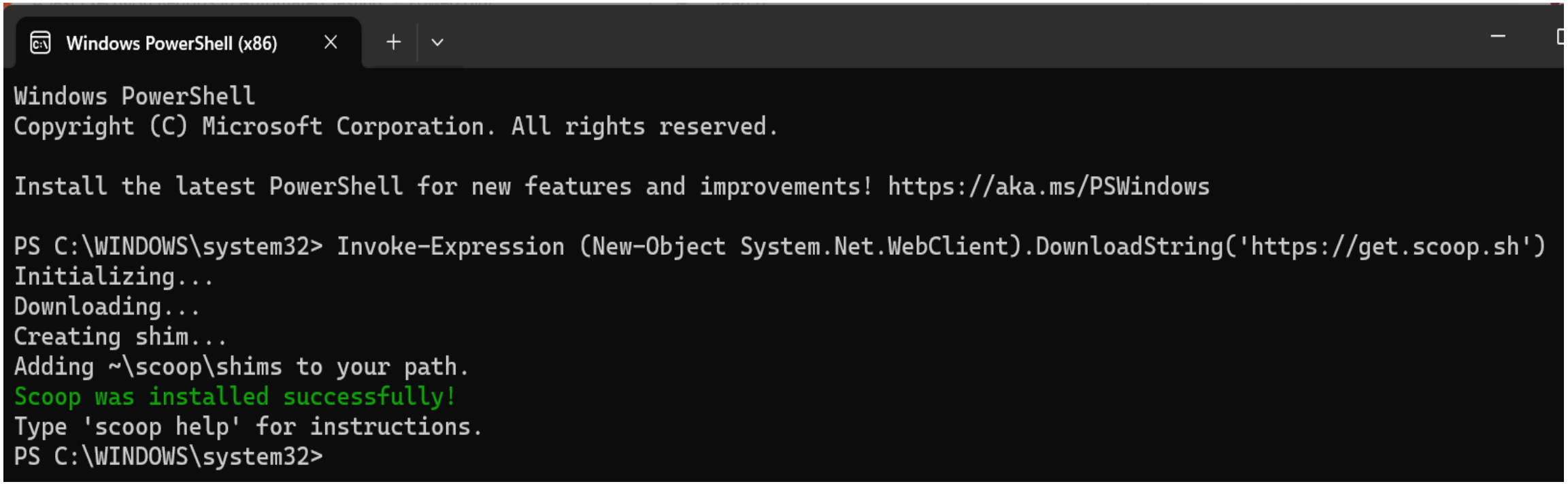
Powershell Set-ExecutionPolicy RemoteSigned -scope CurrentUser



Install Allure

- Run the following command in powershell without administrator rights

Invoke-Expression (New-Object System.Net.WebClient).DownloadString('https://get.scoop.sh')



Install Allure

- To install Allure, download and install Scoop, and then execute in the Powershell:

scoop install allure

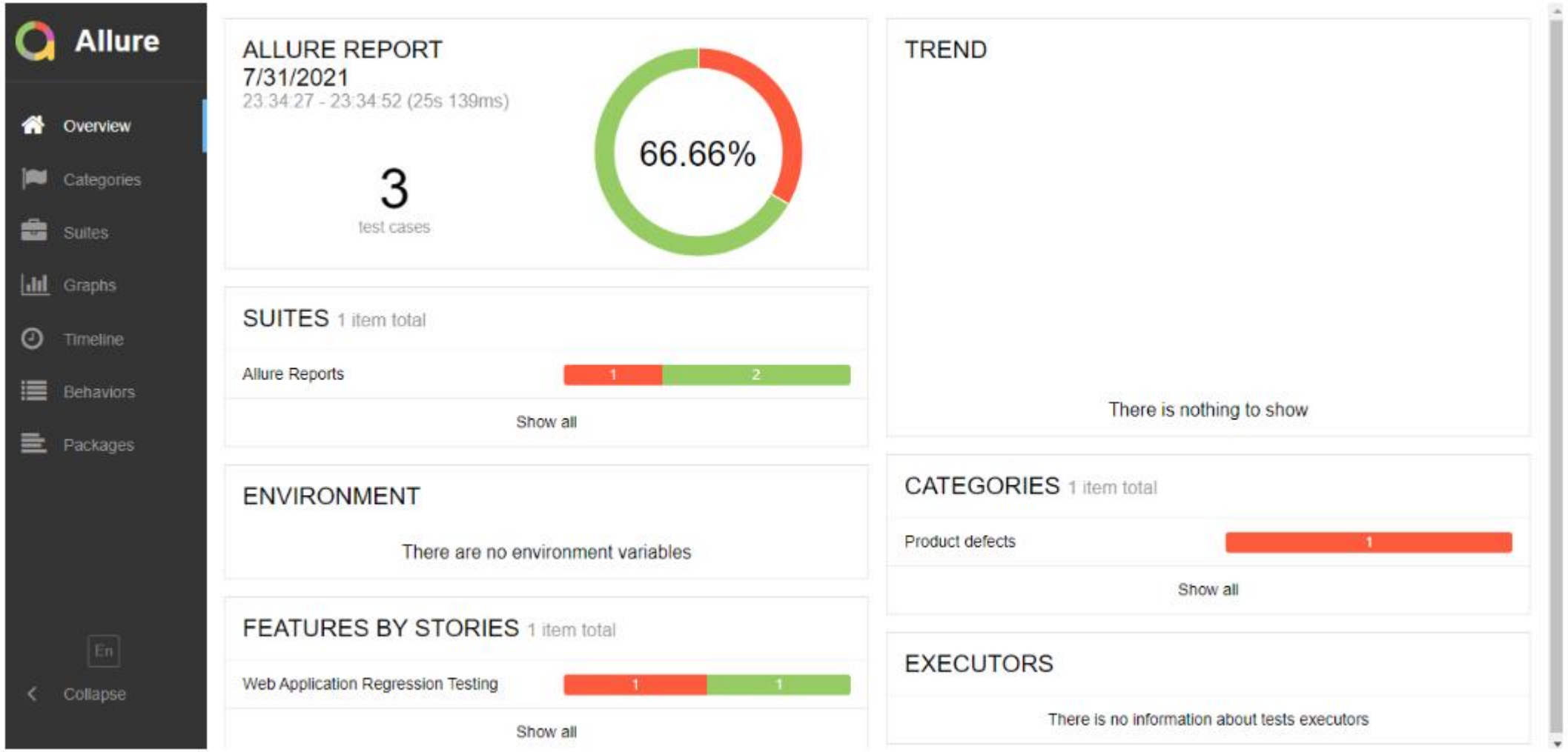
```
PS C:\WINDOWS\system32> scoop install allure
Installing 'allure' (2.36.0) [64bit] from 'main' bucket
allure-2.36.0.zip (23.3 MB) [=========================================================================] 100%
Checking hash of allure-2.36.0.zip ... ok.
Extracting allure-2.36.0.zip ... done.
Linking ~\scoop\apps\allure\current => ~\scoop\apps\allure\2.36.0
Creating shim for 'allure'.
'allure' (2.36.0) was installed successfully!
PS C:\WINDOWS\system32>
```

Install Allure

- To check if you have Allure installed or not, please use the below command in the command line or PowerShell. **allure --version**

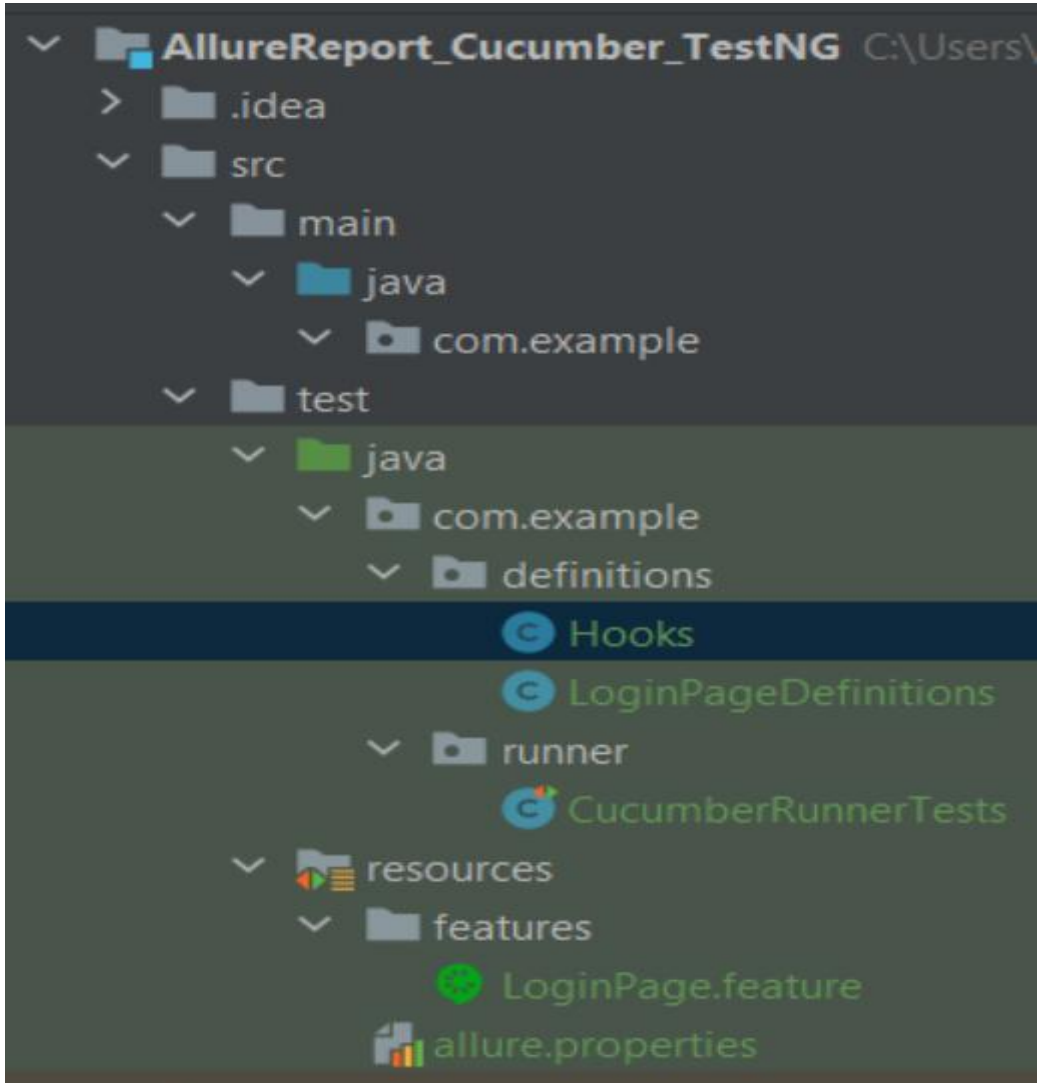
```
PS C:\WINDOWS\system32> allure --version
2.36.0
PS C:\WINDOWS\system32>
```

Sample Allure Report



Generating Allure Report with Cucumber, Selenium and TestNG

Project Structure



Generating Allure Report with Cucumber, Selenium and TestNG

Prerequisite

1. Java 17 installed
2. Maven installed
3. Eclipse or IntelliJ installed
4. Allure installed

Generating Allure Report with Cucumber, Selenium and TestNG

Dependency List – Any Latest Version

1. Selenium – 4.16.1
2. Java 17
3. Cucumber – 7.15.0
4. Maven – 3.9.6
5. Allure Report – 2.25.0
6. Allure Maven – 2.12.0
7. Aspectj – 1.9.21
8. Maven Compiler Plugin – 3.12.1
9. Maven Surefire Plugin – 3.2.3

Step 1: Update the Properties section in Maven pom.xml

```
<properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <cucumber.version>7.15.0</cucumber.version>
    <selenium.version>4.16.1</selenium.version>
    <testng.version>7.9.0</testng.version>
    <maven.compiler.plugin.version>3.12.1</maven.compiler.plugin.version>
    <maven.surefire.plugin.version>3.2.3</maven.surefire.plugin.version>
    <maven.compiler.source.version>17</maven.compiler.source.version>
    <maven.compiler.target.version>17</maven.compiler.target.version>
    <allure.junit4.version>2.25.0</allure.junit4.version>
    <aspectj.version>1.9.21</aspectj.version>
    <allure.version>2.25.0</allure.version>
    <allure.maven>2.12.0</allure.maven>
</properties>
```

Step 2 – Add dependencies to pom.xml

- Add **Cucumber, Selenium, TestNG, Allure-Cucumber, and Allure-TestNG** dependencies to pom.xml (Maven Project).

```
<dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>io.qameta.allure</groupId>
        <artifactId>allure-bom</artifactId>
        <version>${allure.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
```

Step 2 – Add dependencies to pom.xml

- Add **Cucumber, Selenium, TestNG, Allure-Cucumber, and Allure-TestNG** dependencies to pom.xml (Maven Project).

```
<dependencies>

  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-java</artifactId>
    <version>${cucumber.version}</version>
  </dependency>

  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-testng</artifactId>
    <version>${cucumber.version}</version>
    <scope>test</scope>
  </dependency>
```

Step 2 – Add dependencies to pom.xml

```
<!-- Selenium -->
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>${selenium.version}</version>
</dependency>

<!-- TestNG -->
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>${testng.version}</version>
  <scope>test</scope>
</dependency>
```

Step 2 – Add dependencies to pom.xml

```
<!--Allure Cucumber Dependency-->
<dependency>
  <groupId>io.qameta.allure</groupId>
  <artifactId>allure-cucumber7-jvm</artifactId>
  <scope>test</scope>
</dependency>

<!--Allure Reporting Dependency-->
<dependency>
  <groupId>io.qameta.allure</groupId>
  <artifactId>allure-testng</artifactId>
  <scope>test</scope>
</dependency>

</dependencies>
```

Step 3 – Update the Build Section of pom.xml in the Allure Report Project

```
<build>
  <plugins>
   <plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>${maven.compiler.plugin.version}</version>
    <configuration>
     <source>${maven.compiler.source.version}</source>
     <target>${maven.compiler.target.version}</target>
    </configuration>
   </plugin>
```

Step 3 – Update the Build Section of pom.xml in the Allure Report Project

```
<plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>${maven.surefire.plugin.version}</version>
        <configuration>
          <suiteXmlFiles>
            <suiteXmlFile>testng.xml</suiteXmlFile>
          </suiteXmlFiles>
          <argLine>
            -
javaagent:"${settings.localRepository}/org/aspectj/aspectjweaver/${aspectj.version}/aspectjweaver-
${aspectj.version}.jar"
          </argLine>
        </configuration>
```

Step 3 – Update the Build Section of pom.xml in the Allure Report Project

```
<dependencies>
        <dependency>
          <groupId>org.aspectj</groupId>
          <artifactId>aspectjweaver</artifactId>
          <version>${aspectj.version}</version>
          <scope>runtime</scope>
        </dependency>
      </dependencies>
    </plugin>
    <plugin>
      <groupId>io.qameta.allure</groupId>
      <artifactId>allure-maven</artifactId>
      <version>${allure.maven}</version>
      <configuration>
        <reportVersion>${allure.maven}</reportVersion>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Step 4: Create a feature file in src/test/resources

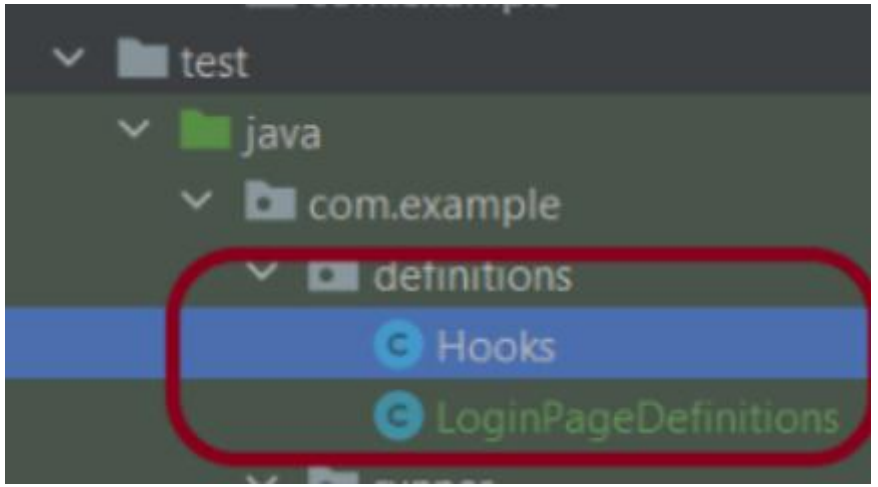
- Create Simple feature file for checking a login in to any application.

```
Feature: Login to HRM Application

  Background:
    Given User is on HRMLogin page "https://opensource-demo.orangehrmlive.com/"

  @ValidCredentials
  Scenario: Login with valid credentials

    When User enters username as "Admin" and password as "admin123"
    Then User should be able to login successfully and new page open
```

Step 4: Create a feature file in src/test/resources

```
@InvalidCredentials
Scenario Outline: Login with invalid credentials

  When User enters username as "<username>" and password as "<password>"
  Then User should be able to see error message "<errorMessage>"

  Examples:
    | username   | password  | errorMessage                  |
    | Admin      | admin12$$ | Invalid credentials           |
    | admin$$    | admin123  | Invalid credentials           |
    | abc123     | xyz$$     | Invalid credentials           |
    | 234        | xyz$$     | Invalid credentials!          |
```

Step 5 – Create the Step Definition class or Glue Code



Step 5.1 – Create the Step Definition class or Glue Code - Hooks.java

```
import io.cucumber.java.After;
import io.cucumber.java.Before;
import io.cucumber.java.Scenario;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

import java.time.Duration;

public class Hooks {
    protected static WebDriver driver;
    public final static int TIMEOUT = 5;

    @Before
    public void setUp() {

        ChromeOptions options = new ChromeOptions();
        options.addArguments("--start-maximized");
        driver = new ChromeDriver(options);
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(TIMEOUT));

    }
```

Step 5.1 – Create the Step Definition class or Glue Code

```
    @After
    public void tearDown(Scenario scenario) {
        try {
            String screenshotName = scenario.getName();
            if (scenario.isFailed()) {
                TakesScreenshot ts = (TakesScreenshot) driver;
                byte[] screenshot = ts.getScreenshotAs(OutputType.BYTES);
                scenario.attach(screenshot, "img/png", screenshotName);
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
        driver.quit();
    }

}
```

Step 5.2 – Create the Step Definition class - LoginPageDefinitions.java

```
import io.cucumber.java.en.Given;
import io.cucumber.java.en.Then;
import io.cucumber.java.en.When;

import org.openqa.selenium.By;
import org.testng.Assert;

public class LoginPageDefinitions {

   Hooks hooks;


    @Given("User is on HRMLogin page {string}")
    public void loginTest(String url) {

        hooks.driver.get(url);

    }
```

Step 5.2 – Create the Step Definition class - LoginPageDefinitions.java

```
@When("User enters username as {string} and password as {string}")
public void goToHomePage(String userName, String passWord) {

    // login to application
    hooks.driver.findElement(By.name("username")).sendKeys(userName);
    hooks.driver.findElement(By.name("password")).sendKeys(passWord);
    hooks.driver.findElement(By.xpath("//*[@class='oxd-form']/div[3]/button")).submit();

    // go the next page
}

@Then("User should be able to login successfully and new page open")
public void verifyLogin() {

    String homePageHeading = hooks.driver.findElement(By.xpath("//*[@class='oxd-topbar-header-
breadcrumb']/h6")).getText();

    //Verify new page - HomePage
    Assert.assertEquals(homePageHeading,"Dashboard");

}
```

Step 5.2 – Create the Step Definition class - LoginPageDefinitions.java

```
    @Then("User should be able to see error message {string}")
    public void verifyErrorMessage(String expectedErrorMessage) {

        String actualErrorMessage = hooks.driver.findElement(By.xpath("//*[@class='orangehrm-login-
error']/div[1]/div[1]/p")).getText();

        // Verify Error Message
        Assert.assertEquals(actualErrorMessage, expectedErrorMessage);

    }

}
```

Step 6 – Create a TestNG Cucumber Runner class

```
import org.testng.annotations.Test;
import io.cucumber.testng.AbstractTestNGCucumberTests;
import io.cucumber.testng.CucumberOptions;


@Test
@CucumberOptions(tags = "", features = {"src/test/resources/features"}, glue =
{"com.example.definitions"},
      plugin = {"pretty","io.qameta.allure.cucumber7jvm.AllureCucumber7Jvm"})

public class CucumberRunnerTests extends AbstractTestNGCucumberTests{

}
```

Step 7 – Create testng.xml for the project

```
<?xml version = "1.0"encoding = "UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name = "Suite1">
   <test name = "Test Demo">
      <classes>
         <class name = "com.example.runner.CucumberRunnerTests"/>
      </classes>
   </test>
</suite>
```

Step 8.1 – Run the Test and Generate Allure Report

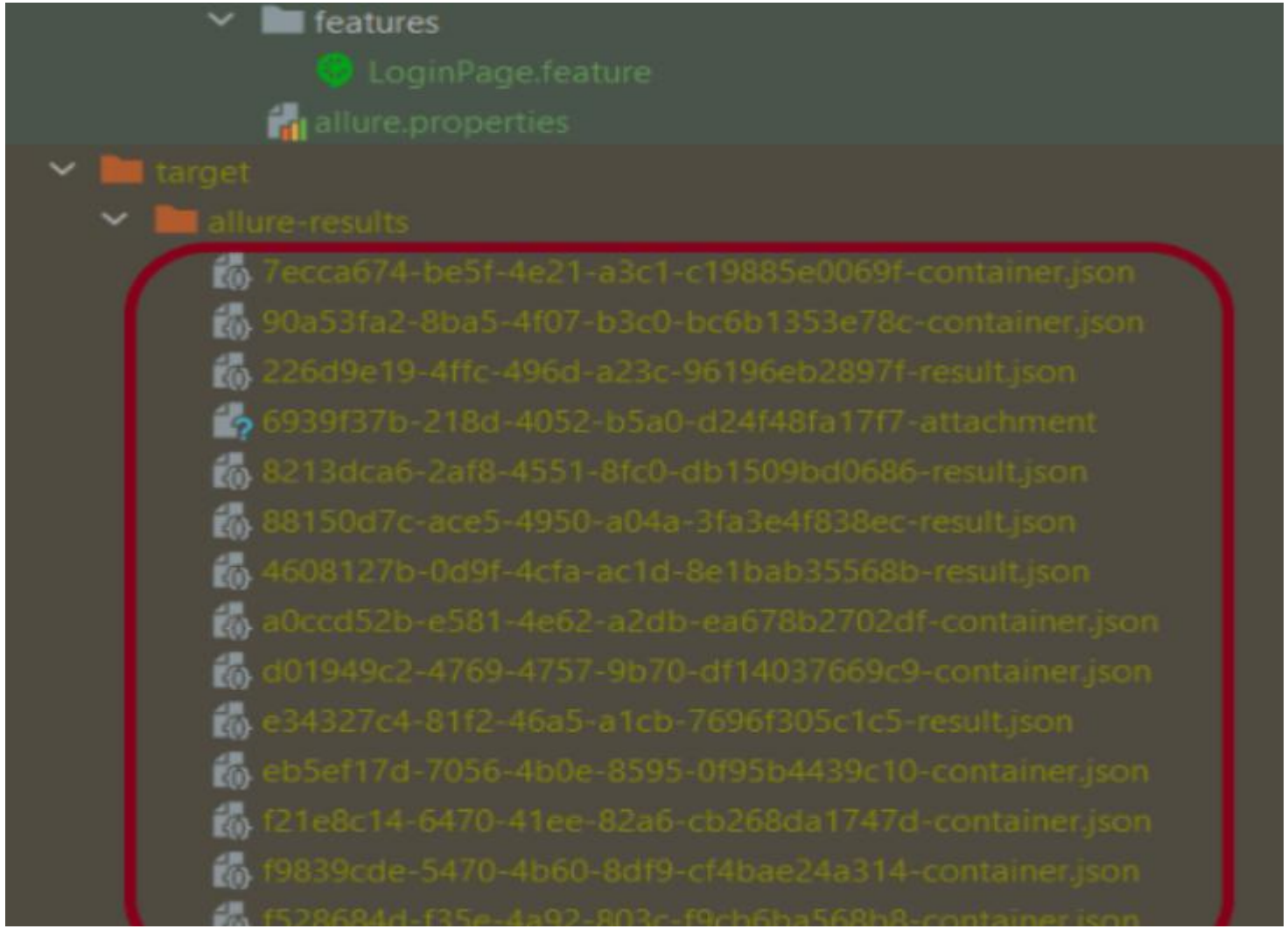
To run the tests, In Command prompt, Go to your project location and use the below command:

mvn clean test

```
PS C:[redacted]Documents\Vibha\Automation\AllureReport_Cucumber_TestNG> mvn clean test
[INFO] Scanning for projects...
[INFO]
[INFO] --------------< com.example:AllureReport_Cucumber_TestNG >--------------
[INFO] Building AllureReport_Cucumber_TestNG 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ AllureReport_Cucumber_TestNG ---
```

- **This will create the allure-results folder with all the test reports within target folder. These files will be used to generate Allure Report.**

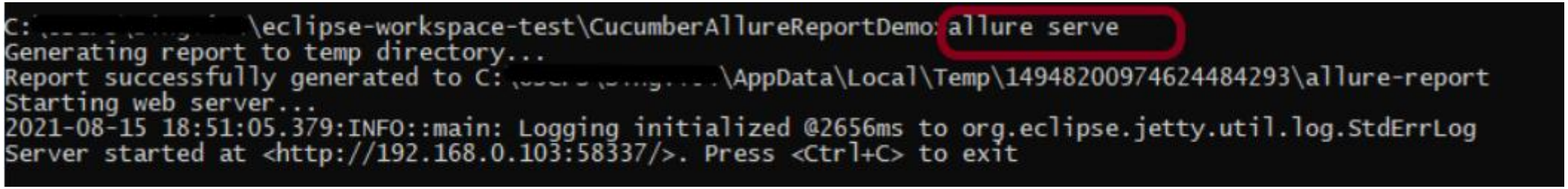
Step 8.1 – Run the Test and Generate Allure Report



Step 8.2 – Serve Allure Report

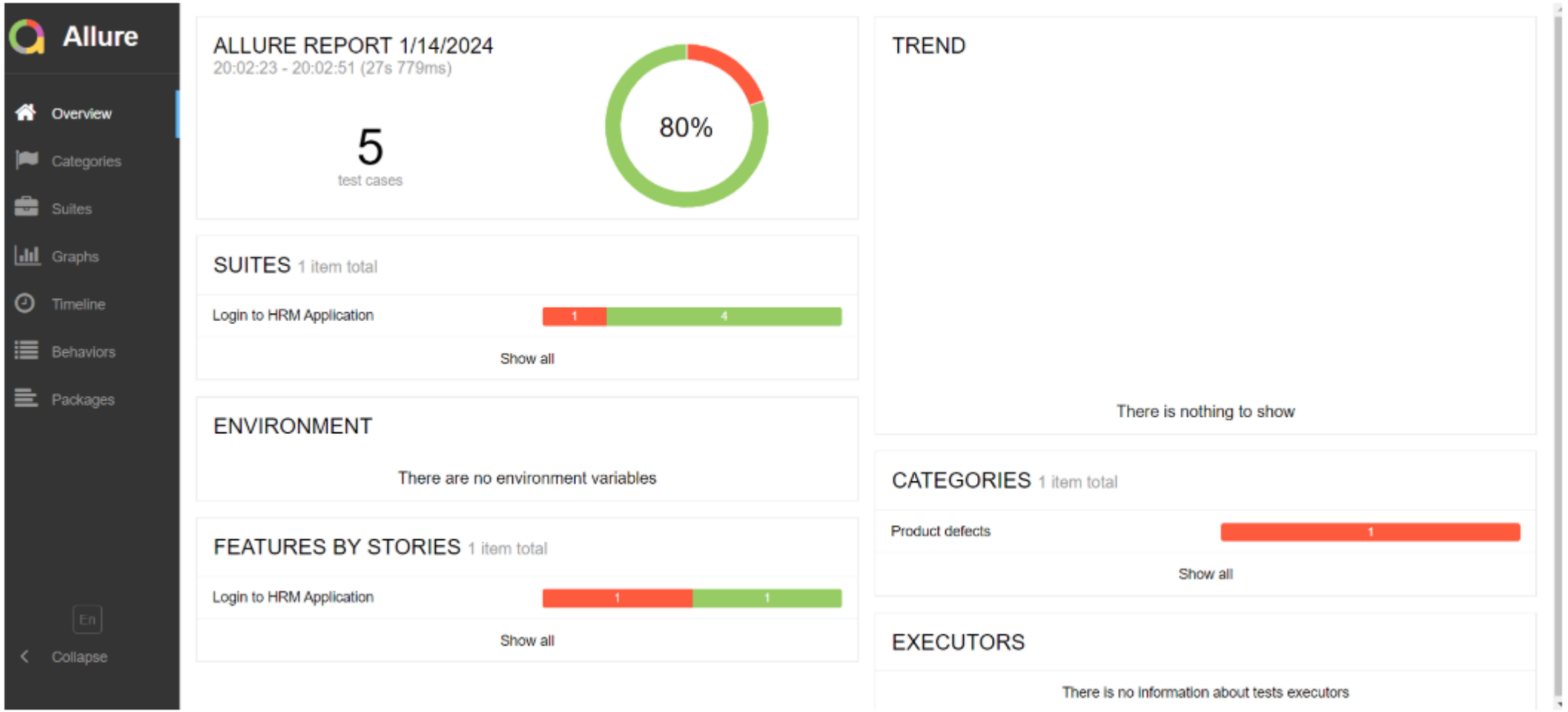
Use the below command to generate the Allure Report:

allure serve

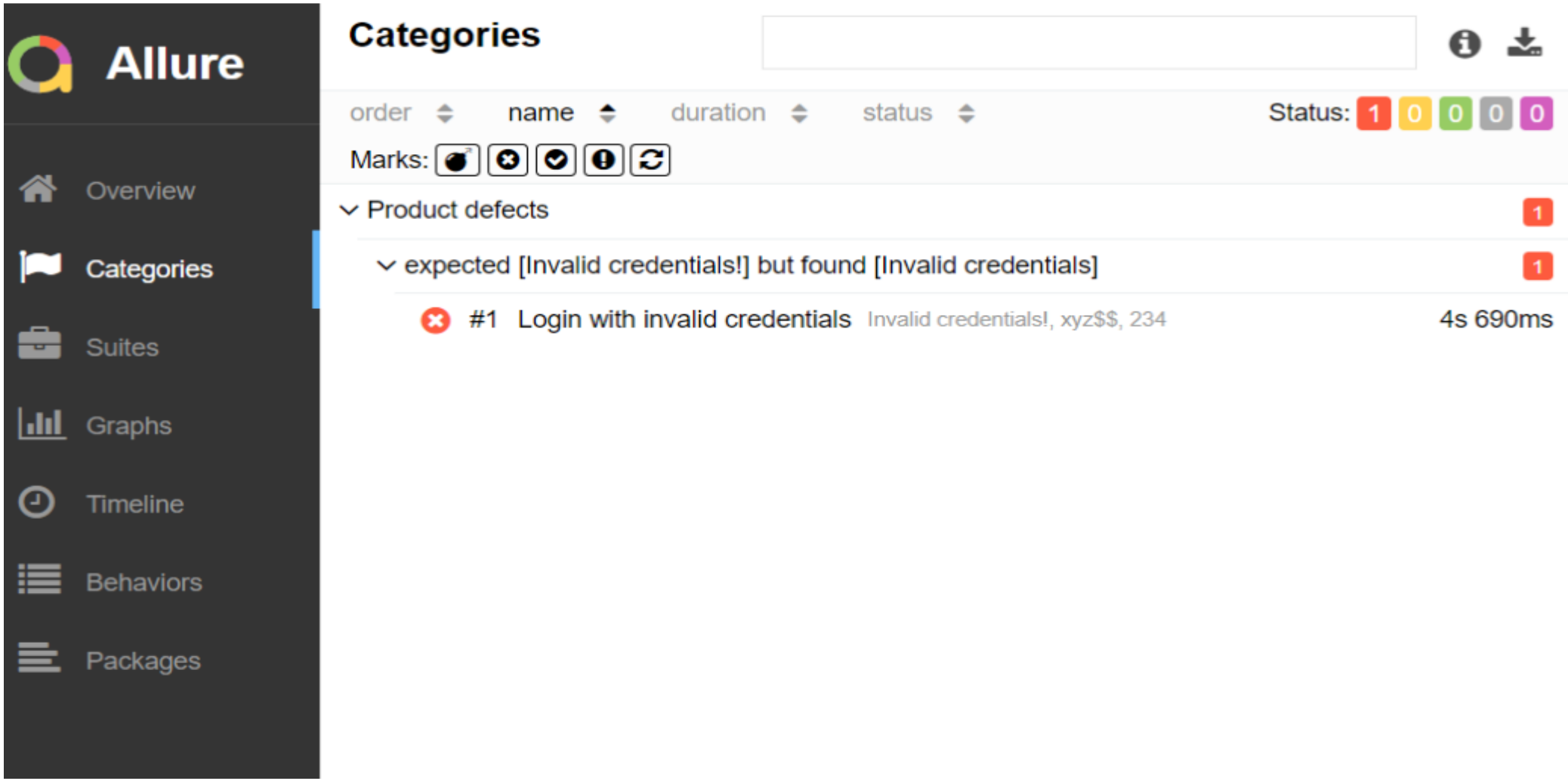


- **This will generate the beautiful Allure Test Report**

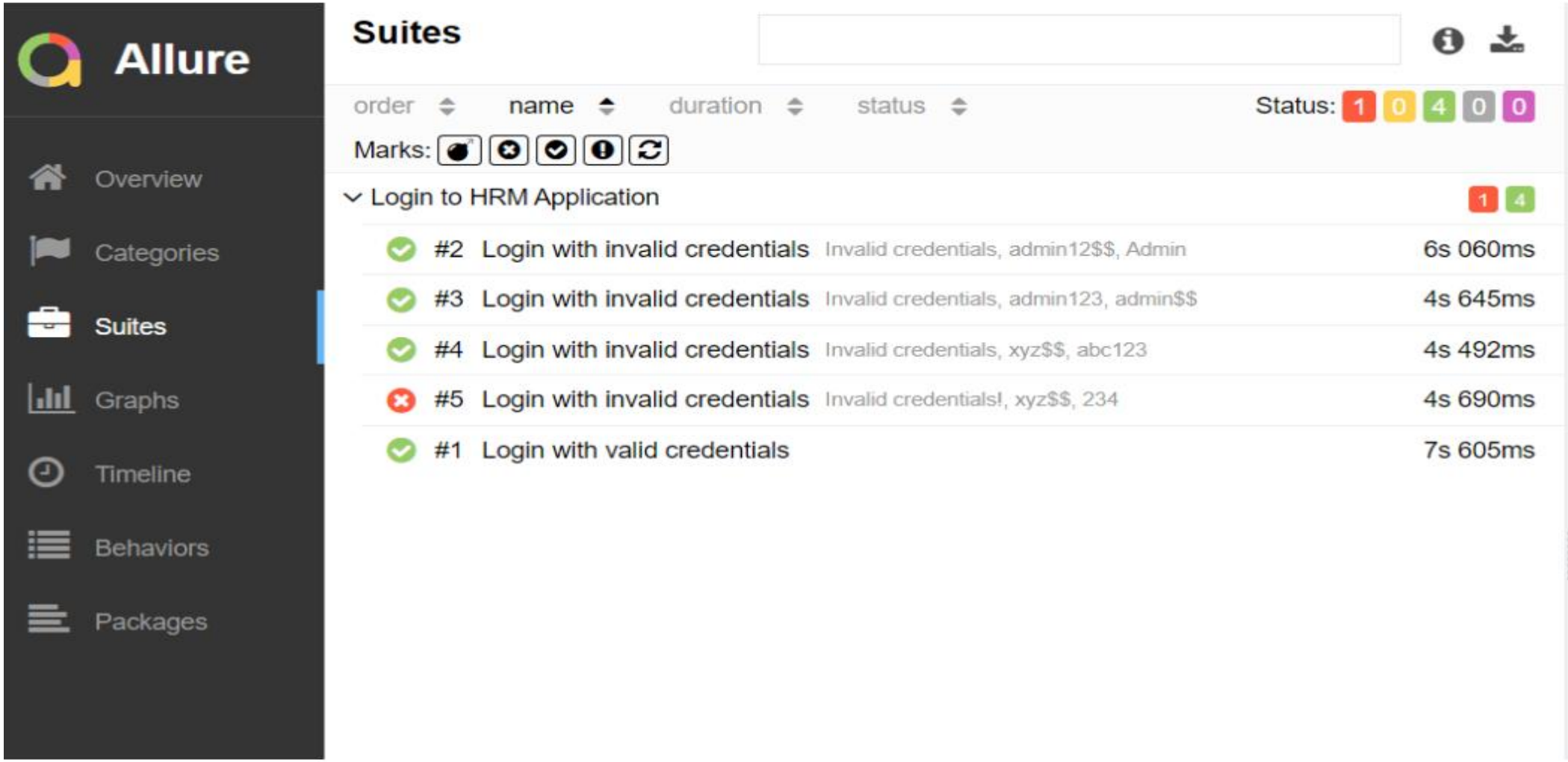
Allure Report Dashboard



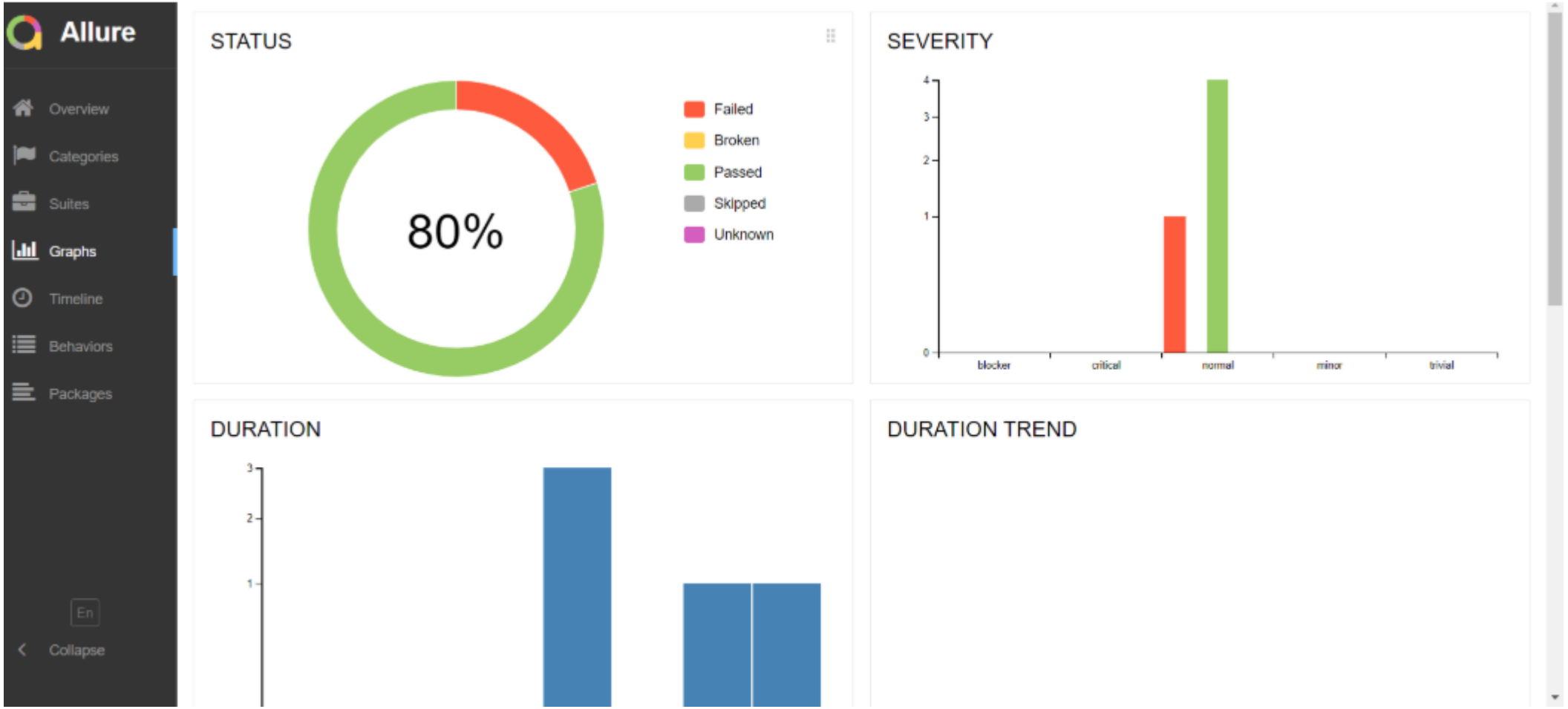
Categories in Allure Report



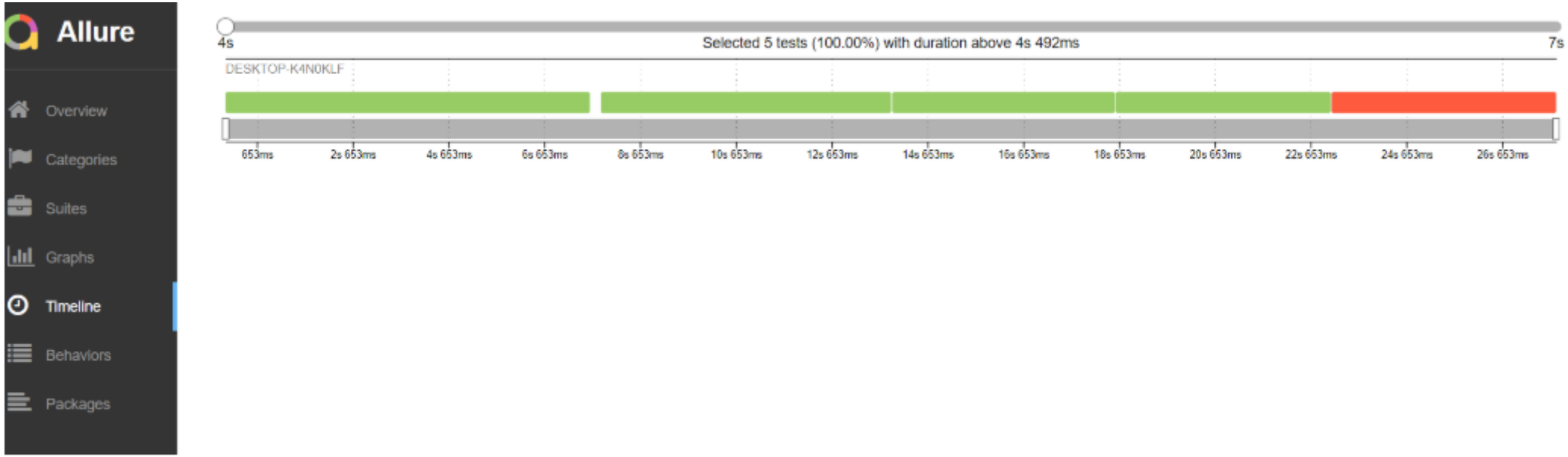
Suites in Allure Report



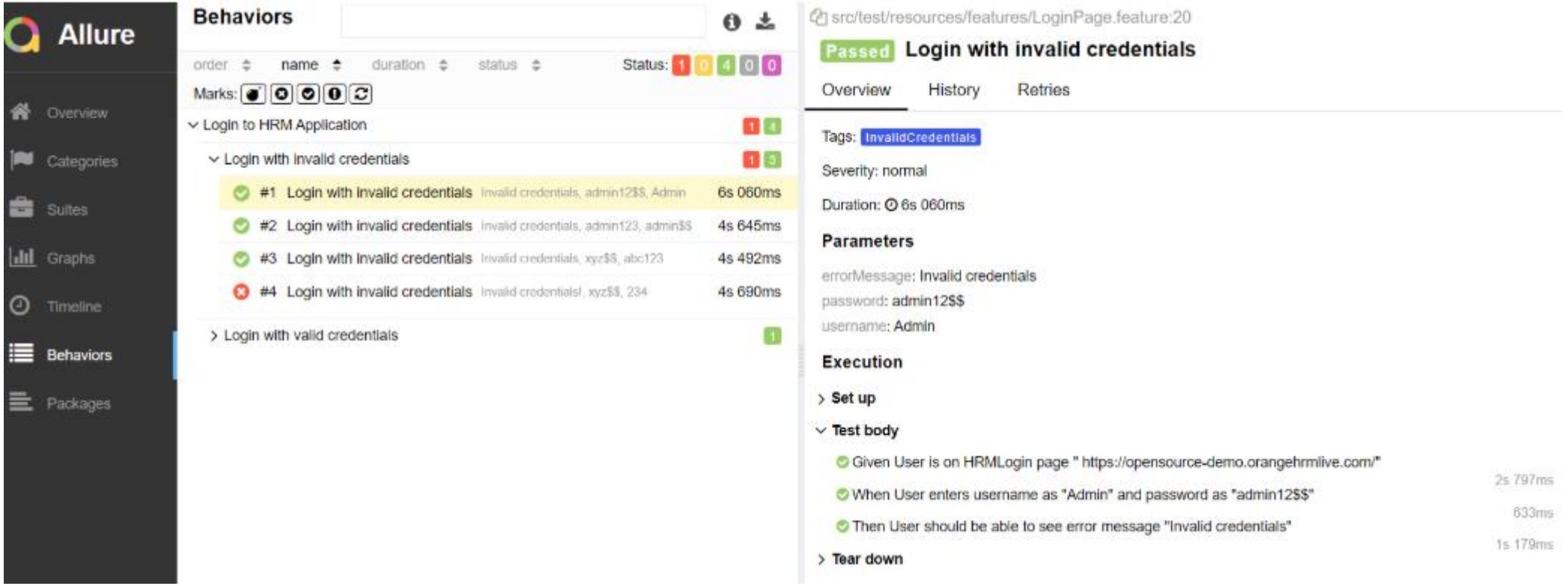
Graphs in Allure Report



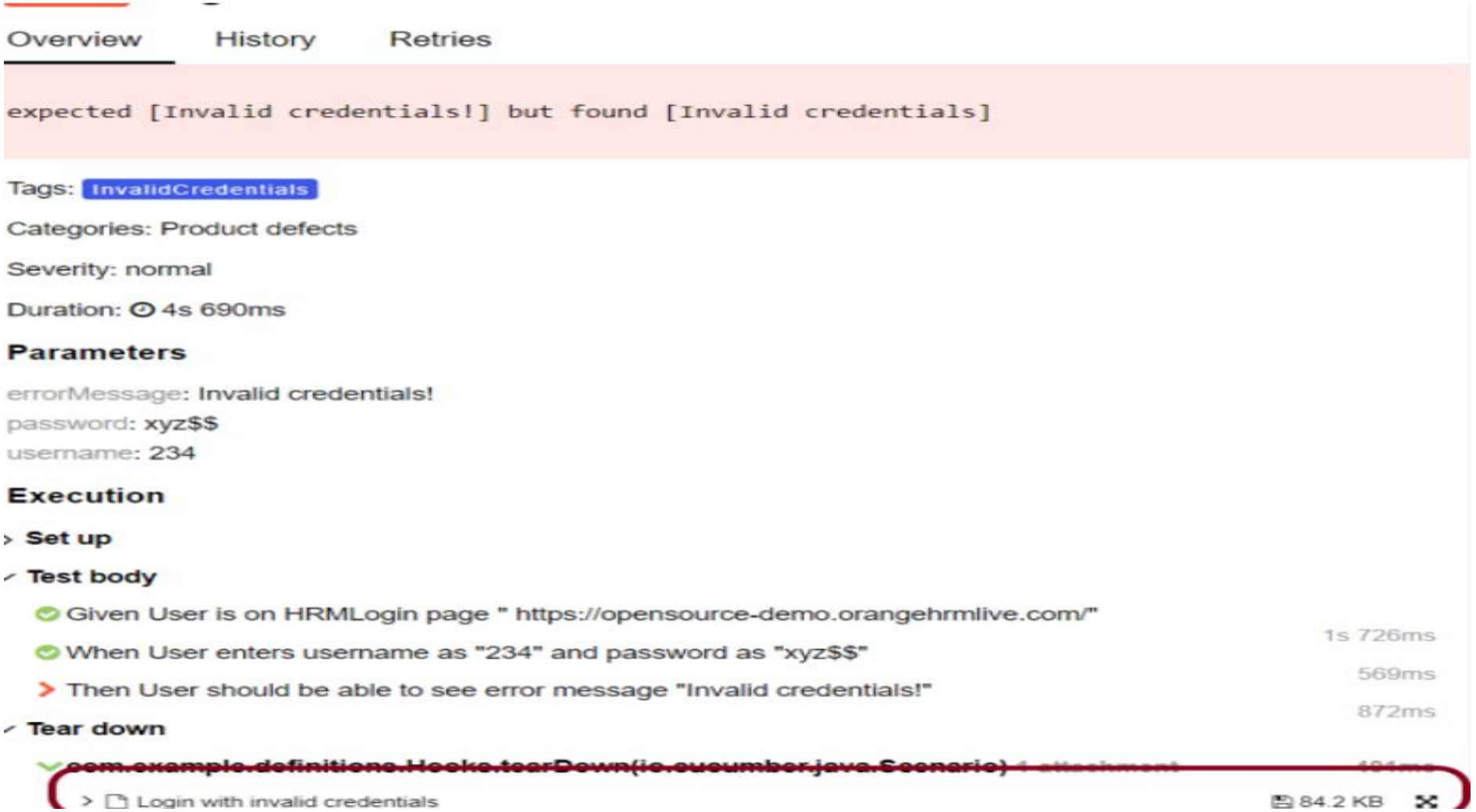
Timeline in Allure Report



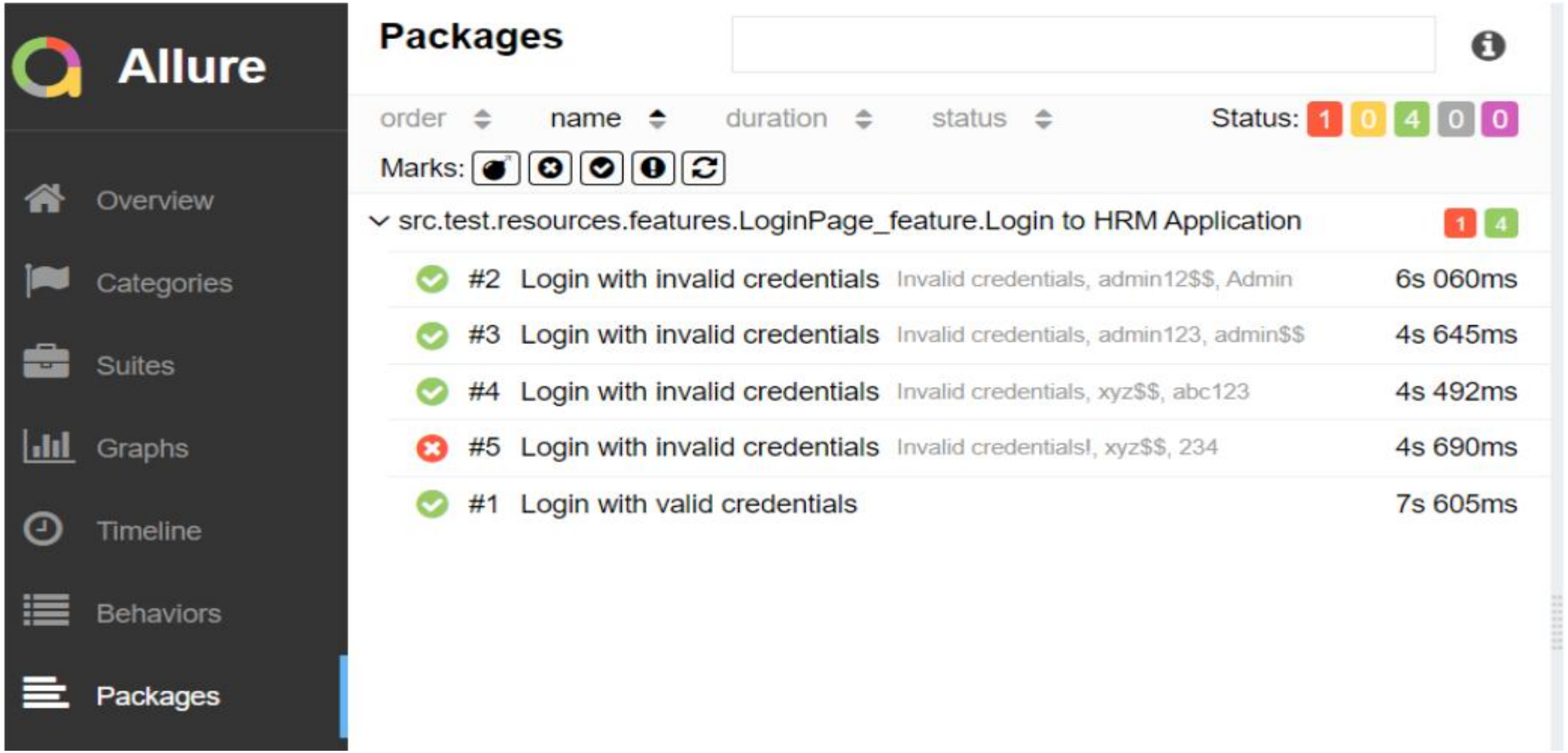
Behaviours of Allure Report



Screenshot attached to the failed test case



Packages in Allure Report




THANK YOU
SmartCliff
Career Mobility Solutions